The Journal of Open Source Software

Uno: a composable framework for nonlinearly constrained optimization

Alexis Montoison [1] **and Charlie Vanaret** [2]

1 Mathematics and Computer Science Division, Argonne National Laboratory, USA **2** Applied Optimization Department, Zuse-Institut Berlin, Germany

DOI: 10.xxxxxx/draft

Software

- Review
- Repository
- Archive

Editor: Open Journals

Reviewers:

- @openjournals

Submitted: 01 January 1970
Published: unpublished

License
Authors of papers retain copyright and release the work under a Creative Commons Attribution 4.0 International License (CC BY 4.0).

DRAFT

Summary

Uno is a composable software framework for nonlinearly constrained optimization written in modern C++. It unifies the workflows of Lagrange-Newton methods, i.e., gradient-based algorithms that iteratively solve the KKT optimality conditions with Newton's method. As of February 2026, Uno supports sequential (convex and nonconvex) quadratic programming, interior-point (barrier) methods, and sequential linear programming.

Uno breaks down optimization algorithms into reusable modular components such as step computation, constraint reformulation, globalization techniques, and acceptance criteria. This allows classical and hybrid methods to be configured and compared within a single framework.

The core C++ implementation separates mathematical abstractions from implementation details through a clear software abstraction layer. Bindings for Julia, Python, C, and Fortran support the use of Uno across scientific computing environments. Precompiled artifacts are available on GitHub. The solver can be accessed directly via `UnoSolver.jl` in Julia or `unopy` in Python.

Statement of need

Nonlinearly constrained optimization is central to engineering, optimal control, machine learning, and scientific modeling. It also plays a central role in mixed-integer nonlinear optimization (MINLP) in which a sequence of continuous relaxations is solved. Existing nonlinear programming solvers are typically monolithic, exposing only parameter tuning rather than structural composition.

This creates limitations for algorithmic research:

- Implementing new methods requires modifying complex legacy code.
- The comparison of algorithmic strategies is difficult to reproduce.
- Hybrid methods combining multiple paradigms are hard to prototype.
- Teaching algorithmic components is challenging.

Uno addresses these gaps by enabling users to assemble algorithms from fundamental building blocks that correspond to mathematical concepts such as step computation, constraint reformulation (penalty or barrier), and globalization techniques. Uno serves both as a practical solver and as a platform for research and education.

State of the field

Nonlinearly constrained optimization solvers typically implement:

- Sequential Quadratic Programming

- Interior-point/barrier methods
- Augmented Lagrangian methods

Existing solvers are robust but structurally rigid, which makes it difficult to exchange components or combine methods. Modeling frameworks simplify the problem formulation but rely on external solvers, and do not expose internal algorithmic structure. Research codes sometimes implement specialized methods, but components are not interchangeable, globalization strategies are hard-wired, and combining paradigms requires significant reimplementation.

Uno differs in that algorithms emerge as possible combinations of strategies within a single composable framework, rather than independent implementations. This enables reproducible research, controlled comparisons, and systematic exploration of hybrid methods. An extended wheel of strategies, organized as layers and ingredients, is shown in Figure 1. Note that all strategies are not available in Uno yet. (TODO: use 2 different colors: one for implemented, one for not yet).

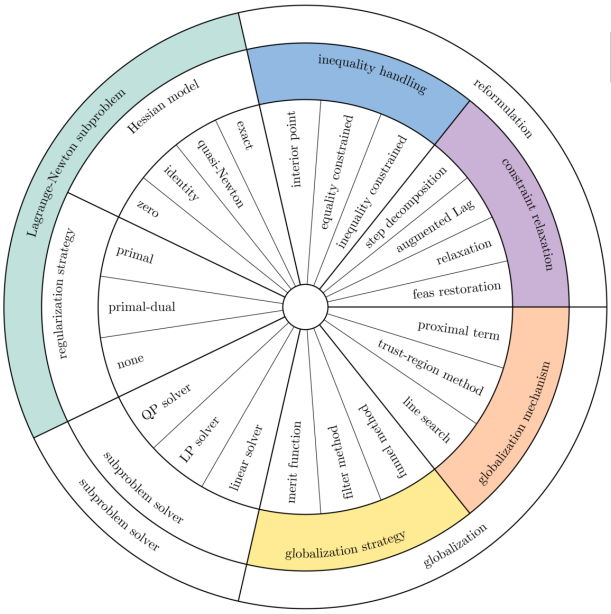

Figure 1: Unification framework: wheel of strategies.

Software design

The architecture of Uno follows a usual object-oriented design in which abstract classes define interfaces that should be implemented by subclasses. For instance, `BacktrackingLineSearch` and `TrustRegionMethod` both inherit from the abstract class `GlobalizationMechanism` and implement its interface.

Research impact statement

...

Uno is also a production-quality solver. Its C++ core is designed to be efficient, and language bindings for Julia, Python, C, and Fortran make it directly usable in scientific and engineering workflows. By representing classical algorithms within a single framework, experiments isolate algorithmic ideas from implementation details. Uno functions as both a state-of-the-art solver and a platform for algorithmic research.

References

DRAFT